

05 | 善用Embedding，我们来给文本分分类

2023-03-28 徐文浩 来自北京

《AI大模型之美》



你好，我是徐文浩。

上一讲里我们看到大模型的确有效。在进行情感分析的时候，我们通过 OpenAI 的 API 拿到的 Embedding，比 T5-base 这样单机可以运行的小模型，效果还是好很多的。

不过，我们之前选用的问题的确有点太简单了。我们把 5 个不同的分数分成了正面、负面和中性，还去掉了相对难以判断的“中性”评价，这样我们判断的准确率高的确是比较好实现的。但如果我们想要准确地预测出具体的分数呢？

利用 Embedding，训练机器学习模型

最简单的办法就是利用我们拿到的文本 Embedding 的向量。这一次，我们不直接用向量之间的距离，而是使用传统的机器学习的方法来进行分类。毕竟，如果只是用向量之间的距离作为衡量标准，就没办法最大化地利用已经标注好的分数信息了。

事实上，OpenAI 在自己的官方教程里也直接给出了这样一个例子。我在这里也放上了相应的 GitHub 的代码 [🔗 链接](#)，你可以去看一下。不过，为了避免 OpenAI 王婆卖瓜自卖自夸，我们也希望能和其他人用传统的机器学习方式得到的结果做个比较。

因此我重新找了一个中文的数据集来试一试。这个数据集是在中文互联网上比较容易找到的一份今日头条的新闻标题和新闻关键词，在 GitHub 上可以直接找到数据，我把 [🔗 链接](#) 也放在这里。用这个数据集的好处是，有人同步放出了预测的实验效果。我们可以拿自己训练的结果和他做个对比。

数据处理，小坑也不少

在训练模型之前，我们要先获取每一个新闻标题的 Embedding。我们通过 Pandas 这个 Python 数据处理库，把对应的文本加载到内存里。接着去调用之前我们使用过的 OpenAI 的 Embedding 接口，然后把返回结果一并存下来就好了。这个听起来非常简单直接，我也把对应的代码先放在下面，不过你先别着急运行。

注：因为后面的代码可能会耗费比较多的 Token 数量，如果你使用的是免费的 5 美元额度的话，可以直接去拿我放在 Github 里的 [🔗 数据文件](#)，用我已经处理好的数据。

 复制代码

```
1 import pandas as pd
2 import tiktoken
3 import openai
4 import os
5
6 from openai.embeddings_utils import get_embedding, get_embeddings
7
8 openai.api_key = os.environ.get("OPENAI_API_KEY")
9
10 # embedding model parameters
11 embedding_model = "text-embedding-ada-002"
12 embedding_encoding = "cl100k_base" # this the encoding for text-embedding-ada-00
13 max_tokens = 8000 # the maximum for text-embedding-ada-002 is 8191
14
15 # import data/toutiao_cat_data.txt as a pandas dataframe
16 df = pd.read_csv('data/toutiao_cat_data.txt', sep='!_', names=['id', 'code', 'ca
17 df = df.fillna("")
18 df["combined"] = (
19     "标题: " + df.title.str.strip() + "; 关键字: " + df.keywords.str.strip()
```

```
20 )
21
22 print("Lines of text before filtering: ", len(df))
23
24 encoding = tiktoken.get_encoding(embedding_encoding)
25 # omit reviews that are too long to embed
26 df["n_tokens"] = df.combined.apply(lambda x: len(encoding.encode(x)))
27 df = df[df.n_tokens <= max_tokens]
28
29 print("Lines of text after filtering: ", len(df))
```

注：这个是加载数据并做一些简单预处理的代码，你可以直接运行。

 复制代码

```
1 # randomly sample 1k rows
2 df_1k = df.sample(1000, random_state=42)
3
4 df_1k["embedding"] = df_1k.combined.apply(lambda x : get_embedding(x, engine=embe
5 df_1k.to_csv("data/toutiao_cat_data_10k_with_embeddings.csv", index=False)
```

注：这个是一条条数据请求 OpenAI 的 API 获取 Embedding 的代码，但是你在运行中会遇到报错。

直接运行这个代码，你多半会遇到一个报错，因为在这个数据处理过程中也是有几个坑的。

第一个坑是 **OpenAI 提供的接口限制了每条数据的长度**。我们这里使用的 text-embedding-ada-002 的模型，支持的长度是每条记录 8191 个 Token。所以我们在实际发送请求前，需要计算一下每条记录有多少 Token，超过 8000 个的需要过滤掉。不过，在我们这个数据集里，只有新闻的标题，所以不会超过这个长度。但是你在使用其他数据集的时候，可能就需要过滤下数据，或者采用截断的方法，只用文本最后 8000 个 Token。

我们在这里，调用了 Tiktoken 这个库，使用了 cl100k_base 这种编码方式，这种编码方式和 text-embedding-ada-002 模型是一致的。如果选错了编码方式，你计算出来的 Token 数量可能和 OpenAI 的不一样。

第二个坑是，如果你直接一条条调用 OpenAI 的 API，很快就会遇到报错。这是因为 **OpenAI 对 API 的调用进行了限速** (Rate Limit)。如果你过于频繁地调用，就会遇到限速的报错。而如果你在报错之后继续持续调用，限速的时间还会被延长。那怎么解决这个问题呢？我习惯选用 backoff 这个 Python 库，在调用的时候如果遇到报错了，就等待一段时间，如果连续报错，就拉长等待时间。通过 backoff 改造的代码我放在了下面，不过这还没有彻底解决问题。

 复制代码

```
1 conda install backoff
```

你需要先安装一下 backoff 库，当然，换成 PIP 安装也是一样的。

 复制代码

```
1 import backoff
2
3 @backoff.on_exception(backoff.expo, openai.error.RateLimitError)
4 def get_embedding_with_backoff(**kwargs):
5     return get_embedding(**kwargs)
6
7 # randomly sample 10k rows
8 df_10k = df.sample(10000, random_state=42)
9
10 df_10k["embedding"] = df_10k.combined.apply(lambda x : get_embedding_with_backoff
11 df_10k.to_csv("data/toutiao_cat_data_10k_with_embeddings.csv", index=False)
```

通过 backoff 库，我们指定了在遇到 RateLimitError 的时候，按照指数级别增加等待时间。

如果你直接运行上面那个代码，大约需要 2 个小时才能处理完 1 万条数据。我们的数据集里有 38 万条数据，真要这么干，需要 3 天 3 夜才能把训练数据处理完，这样显然不怎么实用。这么慢的原因有两个，一个是限速，backoff 只是让我们的调用不会因为失败而终止，但是我还是受到了每分钟 API 调用次数的限制。第二个是延时，因为我们是按照顺序一个个调用 Embedding 接口，每一次调用都要等前一次调用结束后才会发起请求，而不是多条数据并行请求，这更进一步**拖长了处理数据所需要的时间**。

	TEXT & EMBEDDING	CODEX	EDIT	IMAGE
Free trial users	•20 RPM •150,000 TPM	•20 RPM •40,000 TPM	•20 RPM •150,000 TPM	50 images / min
Pay-as-you-go users (first 48 hours)	•60 RPM •250,000 TPM*	•20 RPM •40,000 TPM	•20 RPM •150,000 TPM	50 images / min
Pay-as-you-go users (after 48 hours)	•3,500 RPM •350,000 TPM*	•20 RPM •40,000 TPM	•20 RPM •150,000 TPM	50 images / min

注：你可以点开这个 [🔗 链接](#)，看看目前 OpenAI 对不同模型的限速。

要解决这个问题也不困难，OpenAI 是支持 batch 调用接口的，也就是说，你可以在一个请求里一次批量处理很多个请求。我们把 1000 条记录打包在一起处理，速度就会快很多。我把对应的代码放在下面，你可以试着执行一下，处理这 38 万多条的数据，也就个把小时。不过，你也不能一次性打包太多条记录，因为 OpenAI 的限速不仅仅是针对请求数的，也**限制你每分钟可以处理的 Token 数量**，具体一次打包几条，你可以根据每条数据包含的 Token 数自己测算一下。

 复制代码

```

1 import backoff
2 from openai.embeddings_utils import get_embeddings
3
4 batch_size = 1000
5
6 @backoff.on_exception(backoff.expo, openai.error.RateLimitError)
7 def get_embeddings_with_backoff(prompts, engine):
8     embeddings = []
9     for i in range(0, len(prompts), batch_size):
10         batch = prompts[i:i+batch_size]
11         embeddings += get_embeddings(list_of_text=batch, engine=engine)
12     return embeddings
13
14 # randomly sample 10k rows
15 df_all = df
16 # group prompts into batches of 100
17 prompts = df_all.combined.tolist()

```

```
18 prompt_batches = [prompts[i:i+batch_size] for i in range(0, len(prompts), batch_s
19
20 embeddings = []
21 for batch in prompt_batches:
22     batch_embeddings = get_embeddings_with_backoff(prompts=batch, engine=embeddin
23     embeddings += batch_embeddings
24
25 df_all["embedding"] = embeddings
26 df_all.to_parquet("data/toutiao_cat_data_all_with_embeddings.parquet", index=True
```

最后一个你需要注意的点是，对于这样的大数据集，**不要存储成 CSV 格式**。特别是我们获取到的 Embedding 数据，是很多浮点数，存储成 CSV 格式会把本来只需要 4 个字节的浮点数，都用字符串的形式存储下来，会浪费好几倍的空间，写入的速度也很慢。我在这里**采用了 parquet 这个序列化的格式**，整个存储的过程只需要 1 分钟。

训练模型，看看效果怎么样

数据处理完了，我们就不妨试一试模型训练。如果你担心浪费太多的 API 调用，我把处理好的数据集，放在了我的 [GitHub](#) 上，我把链接也放在了这里，你可以直接下载、使用这个数据集。

```

1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import classification_report, accuracy_score
4
5 training_data = pd.read_parquet("data/toutiao_cat_data_all_with_embeddings.parque
6 training_data.head()
7
8 df = training_data.sample(50000, random_state=42)
9
10 X_train, X_test, y_train, y_test = train_test_split(
11     list(df.embedding.values), df.category, test_size=0.2, random_state=42
12 )
13
14 clf = RandomForestClassifier(n_estimators=300)
15 clf.fit(X_train, y_train)
16 preds = clf.predict(X_test)
17 probas = clf.predict_proba(X_test)
18
19 report = classification_report(y_test, preds)
20 print(report)

```

模型训练的代码也非常简单，考虑到运行时间的因素，我这里直接随机选取了里面的 5 万条数据，4 万条作为训练集，1 万条作为测试集。然后通过最常用的 scikit-learn 这个机器学习工具包里面的随机森林（RandomForest）算法，做了一次训练和测试。在我的电脑上，大概 10 分钟可以跑完，整体的准确率可以达到 84%。

	precision	recall	f1-score	support
1 news_agriculture	0.83	0.85	0.84	495
2 news_car	0.88	0.94	0.91	895
3 news_culture	0.86	0.76	0.81	741
4 news_edu	0.86	0.89	0.87	708
5 news_entertainment	0.71	0.92	0.80	1051
6 news_finance	0.81	0.76	0.78	735
7 news_game	0.91	0.82	0.86	742
8 news_house	0.91	0.86	0.89	450
9 news_military	0.89	0.82	0.85	688
10 news_sports	0.90	0.92	0.91	968
11 news_story	0.95	0.46	0.62	197
12 news_tech	0.82	0.86	0.84	1052
13 news_travel	0.80	0.77	0.78	599
14 news_world	0.83	0.73	0.78	671

16	stock	0.00	0.00	0.00	8
17	accuracy			0.84	10000
18	macro avg	0.80	0.76	0.77	10000
19	weighted avg	0.84	0.84	0.84	10000

随机森林这个算法，虽然效果不错，但是跑起来有些慢。我们接下来用个更简单的逻辑回归 (LogisticRegression) 算法，但我们这次要跑在整个数据集上。一样的，我们拿 80% 作为训练，20% 作为测试。这一次，虽然数据量是刚才 4 万条数据的好几倍，但是时间上却只要 3~4 分钟，而最终的准确率也能达到 86%。

[复制代码](#)

```
1 from sklearn.linear_model import LogisticRegression
2
3 df = training_data
4
5 X_train, X_test, y_train, y_test = train_test_split(
6     list(df.embedding.values), df.category, test_size=0.2, random_state=42
7 )
8
9 clf = LogisticRegression()
10 clf.fit(X_train, y_train)
11 preds = clf.predict(X_test)
12 probas = clf.predict_proba(X_test)
13
14 report = classification_report(y_test, preds)
15 print(report)
```

输出结果：

[复制代码](#)

	precision	recall	f1-score	support
1 news_agriculture	0.86	0.88	0.87	3908
2 news_car	0.92	0.92	0.92	7101
3 news_culture	0.83	0.85	0.84	5719
4 news_edu	0.89	0.89	0.89	5376
5 news_entertainment	0.86	0.88	0.87	7908
6 news_finance	0.81	0.79	0.80	5409
7 news_game	0.91	0.88	0.89	5899
8 news_house	0.91	0.91	0.91	3463
9 news_military	0.86	0.82	0.84	4976
10 news_sports	0.93	0.93	0.93	7611

12	news_story	0.83	0.82	0.83	1308
13	news_tech	0.84	0.86	0.85	8168
14	news_travel	0.80	0.80	0.80	4252
15	news_world	0.79	0.81	0.80	5370
16	stock	0.00	0.00	0.00	70
17	accuracy			0.86	76538
18	macro avg	0.80	0.80	0.80	76538
19	weighted avg	0.86	0.86	0.86	76538

注：下载的数据集所在的测试结果可以看 [🔗这里](#)。

这个结果已经比我们下载数据集的 GitHub 页面里看到的效果好了，那个的准确率只有 85%。

可以看到，通过 OpenAI 的 API 获取到 Embedding，然后通过一些简单的线性模型，我们就能获得很好的分类效果。我们既不需要提前储备很多自然语言处理的知识，对数据进行大量的分析和清理；也不需要搞一块昂贵的显卡，去使用什么深度学习模型。只要 1~2 个小时，我们就能在一个几十万条文本的数据集上训练出一个非常不错的分类模型。

理解指标，学一点机器学习小知识

刚刚我说了，就算你没有机器学习的相关知识，也没有关系，这里我来给你补补课。理解一下上面模型输出的报告是什么意思。报告的每一行都有四个指标，分别是准确率

(Precision)、召回率 (Recall)、F1 分数，以及支持样本量 (Support)。我还是用今日头条的新闻标题这个数据集来解释这些概念。

1. **准确率**，代表模型判定属于这个分类的标题里面判断正确的有多少，有多少真的是属于这个分类的。比如，模型判断里面有 100 个都是农业新闻，但是这 100 个里面其实只有 83 个是农业新闻，那么准确率就是 0.83。准确率自然是越高越好，但是并不是准确率达到 100% 就代表模型全对了。因为模型可能会漏，所以我们还要考虑召回率。
2. **召回率**，代表模型判定属于这个分类的标题占实际这个分类下所有标题的比例，也就是没有漏掉的比例。比如，模型判断 100 个都是农业新闻，这 100 个的确都是农业新闻。准确率已经 100% 了。但是，实际我们一共有 200 条农业新闻。那么有 100 条其实被放到别的类目里面去了。那么在农业新闻这个类目，我们的召回率，就只有 $100/200 = 50\%$ 。

3. 所以模型效果的好坏，既要考虑准确率，又要考虑召回率，综合考虑这两项得出的结果，就是 **F1 分数** (F1 Score) 。F1 分数，是准确率和召回率的调和平均数，也就是 $F1\ Score = 2 / (1/Precision + 1/Recall)$ 。当准确率和召回率都是 100% 的时候，F1 分数也是 1。如果准确率是 100%，召回率是 80%，那么算下来 F1 分数就是 0.88。F1 分数也是越高越好。
4. **支持的样本量**，是指数据里面，实际是这个分类的数据条数有多少。一般来说，数据条数越多，这个分类的训练就会越准确。

分类报告里一个类目占一行，每一行都包含对应的这四个指标，而最下面还有三行数据。这三行数据，是整个拿来测试的数据集，所以对应的支持样本量都是 1 万个。

第一行的 accuracy，只有一个指标，虽然它在 F1 Score 这个列里，但是并不是 F1 分数的意思。而是说，模型总共判断对的分类 / 模型测试的样本数，也就是模型的整体准确率。

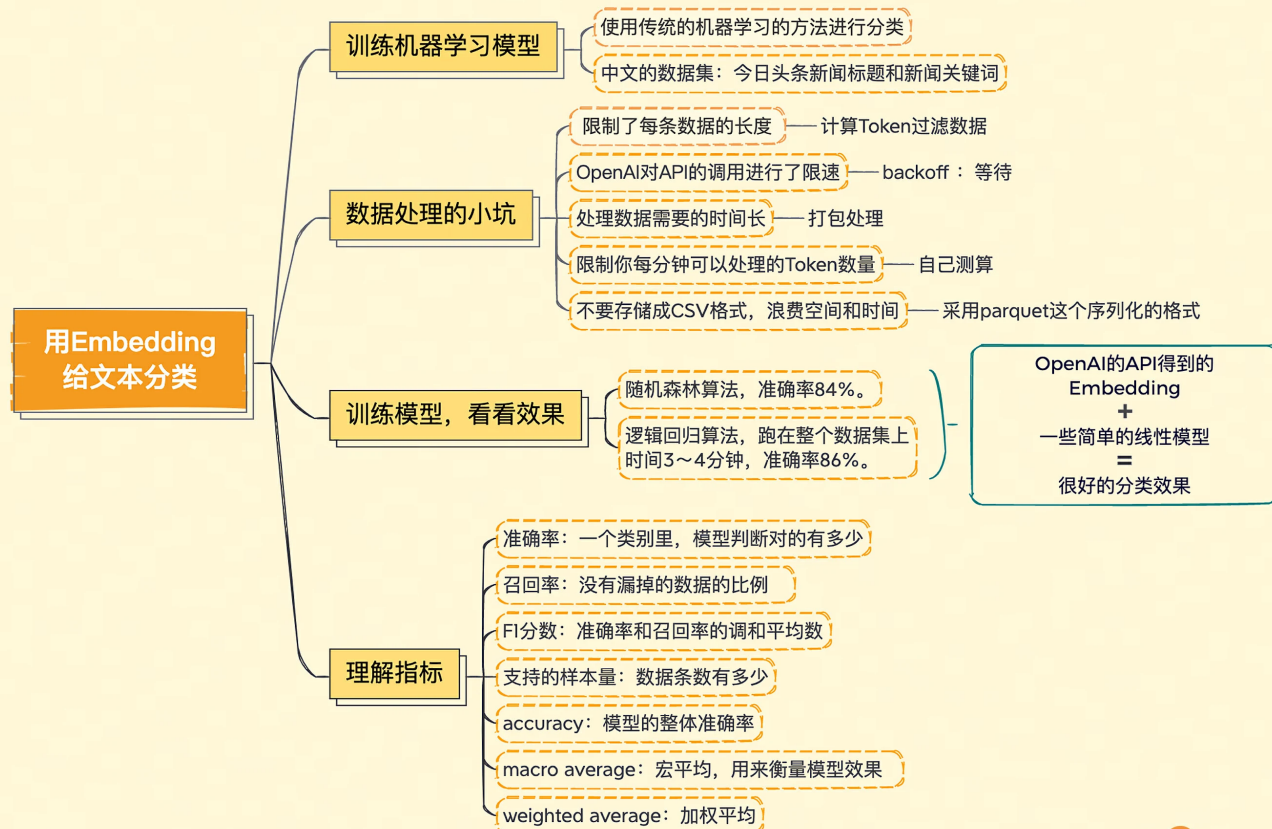
第二行的 macro average，中文名叫做宏平均，宏平均的三个指标，就是把上面每一个分类算出来的指标加在一起平均一下。它主要是在数据分类不太平衡的时候，帮助我们衡量模型效果怎么样。

比如，我们做情感分析，可能 90% 都是正面情感，10% 是负面情感。这个时候，我们预测正面情感效果很好，比如有 90% 的准确率，但是负面情感预测很差，只有 50% 的准确率。如果看整体数据，其实准确率还是非常高的，毕竟负面情感的例子很少。

但是我们的目标可能就是找到有负面情绪的客户和他们沟通、赔偿。那么整体准确率对我们就没有什么用了。而宏平均，会把整体的准确率变成 $(90\% + 50\%) / 2 = 70\%$ 。这就不是一个很好的预测结果了，我们需要进一步优化。宏平均对于数据样本不太平衡，有些类目样本特别少，有些特别多的场景特别有用。

第三行的 weighted average，就是加权平均，也就是我们把每一个指标，按照分类里面支持的样本量加权，算出来的一个值。无论是 Precision、Recall 还是 F1 Score 都要这么按照各个分类加权平均一下。

小结



好了，今天的这一讲到这里就结束了，最后我们来回顾一下。这一讲我们学会了两件事情。

第一件，是怎么利用 OpenAI 的 API 来获取文本的 Embedding。虽然接口不复杂，但是我們也要考虑模型能够接受的最大文本长度，API 本身的限速，以及网络延时带来的问题。

我们分别给出了解决方案，使用 Tiktoken 计算样本的 Token 数量，并进行过滤；在遇到限速问题时通过 backoff 进行指数级别的时间等待；通过一次性批量请求一批数据，最大化我们的吞吐量来解决问题；对于返回的结果，我们可以通过 parquet 这样序列化的方式保存数据，来减少数据尺寸。

第二件，是如何直接利用拿到的 Embedding，简单调用一下 scikit-learn，通过机器学习的方法，进行更准确的分类。我们最终把 Embedding 放到一个简单的逻辑回归模型里，就取得了很不错的分类效果。你学会了吗？

课后练习

这一讲里我们学会了利用 OpenAI 来获取文本的 Embedding，然后通过传统的机器学习方式来进行训练，并评估训练的结果。

我们之前用过 Amazon1000 条食物评论的情感分析数据，在那个数据集里，我们其实已经使用过获取到并保存下来的 Embedding 数据了。那么，你能不能试着在完整的数据集上，训练一个能把从 1 分到 5 分的每一个级别都区分出来的机器学习模型，看看效果怎么样？

整个原始数据集的下载链接我放在 [这里](#)了，欢迎你把你测试出来的结果分享出来，看看和其他人比起来怎么样。另外如果你觉得有收获的话，也欢迎你把这节课分享出去，让更多的人了解用 Embedding 给文本分类的方法。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

精选留言 (35)



曾泽浩

2023-03-28 来自广东

老师，你好！既然都调用open ai的接口，为什么不直接让chatgpt直接返回分类结果呢

作者回复：可以的啊，但是要考虑

1. 成本
2. 效果

在有标注数据的情况下，机器学习的效果通常比让ChatGPT只用先验知识效果还是要好不少的。

共 8 条评论 >



8



良辰美景

2023-04-07 来自上海

“调用了 Tiktoken 这个库，使用了 cl100k_base 这种编码方式，这种编码方式和 text-embedding-ada-002 模型是一致的。如果选错了编码方式，你计算出来的 Token 数量可能和 OpenAI 的不一样。”老师，问你一个学习上的问题，像这种API 文档里这么细节的东西，你是如何获取这些信息的

作者回复: 读一遍API文档, 读一遍OpenAI Cookbook。这是看起来最笨, 其实最节约时间的办法。



7



Toni

2023-03-31 来自瑞士

训练一个能把从 1 分到 5 分的每一个级别都区分出来的机器学习模型.

使用下载的数据 fine_food_reviews_with_embeddings_1k.csv

随机森林模型, 超参数 `n_estimators=100` 下跑的结果

	precision	recall	f1-score	support
1	0.86	0.30	0.44	20
2	1.00	0.38	0.55	8
3	1.00	0.18	0.31	11
4	1.00	0.26	0.41	27
5	0.74	1.00	0.85	134
accuracy			0.76	200
macro avg	0.92	0.42	0.51	200
weighted avg	0.81	0.76	0.71	200

极端随机森林模型 `ExtraTreesClassifier()` 下跑的结果

	precision	recall	f1-score	support
1	1.00	0.20	0.33	20
2	1.00	0.38	0.55	8
3	1.00	0.18	0.31	11
4	1.00	0.26	0.41	27
5	0.73	1.00	0.84	134
accuracy			0.75	200
macro avg	0.95	0.40	0.49	200
weighted avg	0.82	0.75	0.69	200

作者回复: 👍

共 2 条评论 >

👍 4



怡仔

2023-03-28 来自上海

github中的找不到途中列举的数据文件

作者回复: 看一下github的README里给了一个百度网盘的下载方式
对应的数据文件比较大, GitHub里直接放也不太方便下载

共 4 条评论 >

👍 4



麦耀锋

2023-05-05 来自广东

应该这么去理解embedding的使用。以前我们做机器学习的时候（或者相对于深度学习之前的“浅层学习”），不管是有监督还是无监督，一般我们需要做feature engineering，也就是需要data scientist，根据业务、专家领域，对数据提取有用的feature；而在NLP领域，那么就是通过word2vec等各种方式来提取feature。通过openai的embedding接口，实际上就是open ai帮我们做了feature engineering这一步，将文本映射到一个合适的vector space，得到的embedding其实就是文本的feature，所以可以基于这个embedding即feature X来做传统的机器学习

作者回复: 👍

共 2 条评论 >

👍 3



lala

2023-04-03 来自北京

现在chatGPT是通用的问答应用。请问如何根据ChatGPT结合公司业务的知识库和商品信息，如何打造服务客户和内部顾问的机器人？比如根据用户问题，推荐对应产品给用户。根据内部知识库，回答内部同事关于产品使用，运营信息相关的问题呢。谢谢

作者回复: 看最新上线的10, 11讲，以及后面会上线的14-17讲



👍 3



Toni

2023-03-30 来自瑞士

在执行这段代码时会报ParserWarning

```
df = pd.read_csv('data/toutiao_cat/toutiao_cat_data.txt', sep='!_', names=['id', 'code', 'category', 'title', 'keywords'])
```

ParserWarning: Falling back to the 'python' engine because the 'c' engine does not support regex separators (separators > 1 char and different from '\s+' are interpreted as regex); you can avoid this warning by specifying engine='python'.

原因是 'c' 引擎不支持分隔符sep='!_'表达式，可指定 engine='python' 来避免此警告。

代码如下

```
df = pd.read_csv('data/toutiao_cat/toutiao_cat_data.txt', engine='python', sep='!_', names=['id', 'code', 'category', 'title', 'keywords'])
```

作者回复: 👍 我本机是默认会退回到用python引擎来分割，感谢。



👍 2



R9Go

2023-05-02 来自韩国

https://github.com/xuwenhao/geektime-ai-course/blob/main/data/20_newsgroup_with_embedding.parquet

这个数据是不是不全？

作者回复: 应该是全的，或者可以去README里面列出来的百度网盘下载

链接: <https://pan.baidu.com/s/1Cl0eFNLOkQqquf9ls0trEw> 提取码: jvr4



👍 1



Geek_512735

2023-04-17 来自北京

老师，请利用ChatGPT embedding的优势在什么地方？

作者回复: 效果好，语义表达能力强，成本也不高。主要缺点是需要把数据发送给OpenAI。



👍 1



Bank

2023-04-08 来自北京

在读数据的时候报了这个错，请问如何解决呢？内存应该是足够的

```
-----
ArrowMemoryError                                Traceback (most recent call last)
Cell In[6], line 6
      3 from sklearn.model_selection import train_test_split
      4 from sklearn.metrics import classification_report, accuracy_score
----> 6 training_data = pd.read_parquet("D:/work/data/toutiao_cat_data_all_with_embeddings.parquet")
      7 training_data.head()
      9 df = training_data.sample(50000, random_state=42)

. . . . .
-> 2517 table = self._dataset.to_table(
    2518     columns=columns, filter=self._filter_expression,
    2519     use_threads=use_threads
    2520 )
    2522 # if use_pandas_metadata, restore the pandas metadata (which gets
    2523 # lost if doing a specific `columns` selection in to_table)
    2524 if use_pandas_metadata:

File D:\anaconda3\lib\site-packages\pyarrow\_dataset.pyx:332, in pyarrow._dataset.Dataset.to_table()

File D:\anaconda3\lib\site-packages\pyarrow\_dataset.pyx:2661, in pyarrow._dataset.Scanner.to_table()

File D:\anaconda3\lib\site-packages\pyarrow\error.pxi:144, in pyarrow.lib.pyarrow_internal_check_status()

File D:\anaconda3\lib\site-packages\pyarrow\error.pxi:117, in pyarrow.lib.check_status()

ArrowMemoryError: malloc of size 134217728 failed
```

作者回复: 134217728 是128MB内存空间，看起来应该不会环境哪里做了限制？

即使在Windows下，我也推荐使用WSL2的Linux子系统来运行这些代码。



1



四月. 🇨🇳

2023-04-05 来自北京

请问一下老师，既然已经按照batch_size划分出来了prompt_batches，为什么在get_embeddings_with_backoff函数中还要划分一次batch呢？是不是重复了？

作者回复: 看了一下的是可以不用。原来是想做成两层batch_size的，就是1000个batch扔给get_embeddings_with_backoff，然后里面再100个一个batch这样，不过后来发现没有必要。



1



Geek_33785c

2023-04-03 来自上海

老师请问如果把时序数据，比如用户行为序列作为输入给openAI 的 embedding，根据行为判断用户情感的效果会好吗？

作者回复: 我没有试过，你可以试一下，如果有结果欢迎分享。我猜如果一般人能判断出来，它就能判断出来。



1



Oli张帆

2023-03-31 来自新加坡

我对embedding的一些理解，就是在与用户交互时，它可以帮助你不在消耗token的情况下，把你当前需要的最相关的context信息从你的知识库提取出来。我个人觉得这个是目前，把自己的专有知识库和人工智能结合起来的最高效性价比最高的一个办法。不知道我这个看法有没有有一定道理，请老师和大家来指正。

共 3 条评论 >

1



Geek_513b7c

2023-03-28 来自吉林

老师，你能解释一下在ai中向量是什么意思吗？这几节看的云里雾里的

作者回复: 可以认为，就是把文本，用一组数字来表示，这组数字表示了这段文本在高维空间里面的

坐标。两段文本相似，就是他们坐标之间比较近。

共 2 条评论 >

👍 1



xbc

2023-03-28 来自海南

from openai.embeddings_utils import get_embeddings 用的是 text-similarity-davinci-001 吧.

openai.Embedding.create 可以选择用 text-embedding-ada-002

作者回复: 默认embedding是 text-similarity-davinci-001
第8讲，会列举模型差别

text-similarity-davinci-001 效果比 text-embedding-ada-002 在相似度比较上其实更好，但是太贵了。



👍 1



Geek_d7a1f9

2023-05-31 来自北京

老师，为什么 chatgpt 的 embedding 需要通过 api 调用，如果我们知道具体算法，实现一个库本地计算不是更好吗



石云升

2023-05-23 来自广东

琢磨了一下才理解召回率的意思。举个例子，如果我预测了200个邮件，其中80个我预测为垃圾邮件，其他120个为正常邮件。但实际上我预测的这80个里，有70个是真的垃圾邮件。那么预测垃圾邮件准确率就是70/80，另外120个正常邮件里，还有10个垃圾邮件，那么垃圾邮件的召回率就是70/80（总的垃圾邮件数量）



Geek_378f83

2023-05-21 来自美国

老师，同学们好，请教一下：

colab 中运行时报错： /lib/x86_64-linux-gnu/libstdc++.so.6: version `GLIBCXX_3.4.29' not found (required by /usr/local/lib/python3.10/site-packages/sentencepiece/_sentencepiece.cp

ython-310-x86_64-linux-gnu.so); 需要在colab中怎么解决?
询问gpt后目前仍未解决, 求助

作者回复: 其实ChatGPT回答不了的问题, 还是可以Google或者Stackoverflow搜索一下, 参看

<https://stackoverflow.com/questions/67720198/how-to-upgrade-libstdc-so-on-colab-runtime>

Colab的确环境搭建不统一, 会给大家造成点困扰



Geek_93970d

2023-05-20 来自北京

```
import pandas as pd
import numpy as np
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
# 换成正确的路径
df_all = pd.read_csv("../data/fine_food_reviews_with_embeddings_1k.csv")
df_all["embedding"] = df_all.embedding.apply(eval).apply(np.array)

# 其实没必要转换成 parquet
df_all.to_parquet("fine_food_reviews_with_embeddings.parquet", index=True)
training_data = pd.read_parquet("fine_food_reviews_with_embeddings.parquet")
training_data.head()

#df = training_data.sample(50000, random_state=42)
#df = df_all
df = training_data

X_train, X_test, y_train, y_test = train_test_split(
    list(df.embedding.values), df.Score, test_size=0.2, random_state=42
)

clf = RandomForestClassifier(n_estimators=300)
clf.fit(X_train, y_train)
```

```
preds = clf.predict(X_test)
probas = clf.predict_proba(X_test)

report = classification_report(y_test, preds)
print(report)
```

-----结果如下-----

	precision	recall	f1-score	support
1	0.89	0.40	0.55	20
2	1.00	0.38	0.55	8
3	1.00	0.18	0.31	11
4	1.00	0.26	0.41	27
5	0.75	1.00	0.86	134
accuracy			0.77	200
macro avg	0.93	0.44	0.53	200
weighted avg	0.82	0.77	0.72	200

作者回复: 👍



Geek_93970d

2023-05-19 来自北京

data/toutiao_cat_data_all_with_embeddings.parquet

这个文件找不到

作者回复: <https://github.com/xuwenhao/geektime-ai-course> 去README里面找一下百度网盘, 下载对应数据文件

链接: <https://pan.baidu.com/s/1Cl0eFNLOkQqquf9ls0trEw> 提取码: jvr4

